◆ Beginner Level (Basics) — Q1 to Q10

(Python with **built-in** + **manual**)

```
# 1. Reverse a string
def reverse_string_builtin(s):
    return s[::-1]

def reverse_string_manual(s):
    rev = ""
    for ch in s:
        rev = ch + rev
    return rev


# 2. Check if a string is palindrome
def is_palindrome_builtin(s):
    return s == s[::-1]

def is_palindrome_manual(s):
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True


# 3. Find length of a string
def string_length_builtin(s):
    return len(s)

def string_length_manual(s):
    count = 0
    for _ in s:
        count += 1
    return count


# 4. Count vowels and consonants
def count_vc_builtin(s):
    vowels = set("aeiouAEIOU")
    v = sum(1 for ch in s if ch in vowels)
    c = sum(1 for ch in s if ch.isalpha() and ch not in vowels)
    return v, c

def count_vc_manual(s):
    vowels = "aeiouAEIOU"
    v = c = 0
    for ch in s:
        if ('a' <= ch <= 'z') or ('A' <= ch <= 'Z'):
            if ch in vowels:
                v += 1
            else:
                c += 1
```

```
    return v, c


# 5. Convert to uppercase/lowercase
def to_upper_builtin(s):
    return s.upper()

def to_lower_builtin(s):
    return s.lower()

def to_upper_manual(s):
    result = ""
    for ch in s:
        if "a" <= ch <= "z":
            result += chr(ord(ch) - 32)
        else:
            result += ch
    return result

def to_lower_manual(s):
    result = ""
    for ch in s:
        if "A" <= ch <= "Z":
            result += chr(ord(ch) + 32)
        else:
            result += ch
    return result


# 6. Remove whitespaces
def remove_spaces_builtin(s):
    return s.replace(" ", "")

def remove_spaces_manual(s):
    result = ""
    for ch in s:
        if ch != " ":
            result += ch
    return result


# 7. Count frequency of each character
def char_freq_builtin(s):
    from collections import Counter
    return dict(Counter(s))

def char_freq_manual(s):
    freq = {}
    for ch in s:
        if ch not in freq:
            freq[ch] = 1
        else:
            freq[ch] += 1
    return freq


# 8. First non-repeating character
def first_non_repeat_builtin(s):
    from collections import Counter
    freq = Counter(s)
    for ch in s:
```

```
        if freq[ch] == 1:
            return ch
    return None

def first_non_repeat_manual(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    for ch in s:
        if freq[ch] == 1:
            return ch
    return None


# 9. Replace spaces with %20 (URLify)
def urlify_builtin(s):
    return s.replace(" ", "%20")

def urlify_manual(s):
    result = ""
    for ch in s:
        if ch == " ":
            result += "%20"
        else:
            result += ch
    return result


# 10. Compare two strings
def compare_builtin(s1, s2):
    return s1 == s2

def compare_manual(s1, s2):
    if string_length_manual(s1) != string_length_manual(s2):
        return False
    for i in range(string_length_manual(s1)):
        if s1[i] != s2[i]:
            return False
    return True
```

🔻 Substrings / Subsequence Problems (Q11–Q20)

11. Find all substrings of a string

```
def all_substrings_builtin(s):
    return [s[i:j] for i in range(len(s)) for j in range(i+1, len(s)+1)]

def all_substrings_manual(s):
    substrings = []
    n = 0
    while n < len(s):
        m = n + 1
        while m <= len(s):
            substrings.append(s[n:m])
            m += 1
        n += 1
    return substrings
```

12. Longest substring without repeating characters

```
def longest_unique_substring_builtin(s):
    seen, start, max_len = {}, 0, 0
    for i, ch in enumerate(s):
        if ch in seen and seen[ch] >= start:
            start = seen[ch] + 1
        seen[ch] = i
        max_len = max(max_len, i - start + 1)
    return max_len

def longest_unique_substring_manual(s):
    max_len = 0
```

```
    for i in range(len(s)):
        visited = {}
        j = i
        while j < len(s) and s[j] not in visited:
            visited[s[j]] = True
            max_len = max(max_len, j - i + 1)
            j += 1
    return max_len


# 13. Longest palindromic substring
def longest_palindrome_builtin(s):
    res = ""
    for i in range(len(s)):
        for j in range(i, len(s)):
            sub = s[i:j+1]
            if sub == sub[::-1] and len(sub) > len(res):
                res = sub
    return res

def longest_palindrome_manual(s):
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1; r += 1
        return s[l+1:r]
    res = ""
    for i in range(len(s)):
        p1 = expand(i, i)
        p2 = expand(i, i+1)
        res = p1 if len(p1) > len(res) else res
        res = p2 if len(p2) > len(res) else res
```

```
    return res


# 14. Longest common prefix
def longest_common_prefix_builtin(strs):
    if not strs: return ""
    prefix = min(strs)
    for s in strs:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
    return prefix

def longest_common_prefix_manual(strs):
    if not strs: return ""
    prefix = strs[0]
    for i in range(1, len(strs)):
        j = 0
        while j < len(prefix) and j < len(strs[i]) and prefix[j] == strs[i][j]:
            j += 1
        prefix = prefix[:j]
    return prefix


# 15. Longest Common Subsequence (DP)
def lcs_builtin(s1, s2):
    from functools import lru_cache
    @lru_cache(None)
    def helper(i, j):
        if i == len(s1) or j == len(s2): return 0
        if s1[i] == s2[j]:
            return 1 + helper(i+1, j+1)
```

```
        return max(helper(i+1, j), helper(i, j+1))
    return helper(0, 0)

def lcs_manual(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(n-1, -1, -1):
        for j in range(m-1, -1, -1):
            if s1[i] == s2[j]:
                dp[i][j] = 1 + dp[i+1][j+1]
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j+1])
    return dp[0][0]



# 16. Edit Distance (Levenshtein)
def edit_distance_builtin(s1, s2):
    import Levenshtein
    return Levenshtein.distance(s1, s2)  # requires python-Levenshtein lib

def edit_distance_manual(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(n+1): dp[i][0] = i
    for j in range(m+1): dp[0][j] = j
    for i in range(1, n+1):
        for j in range(1, m+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
```

```
    return dp[n][m]


# 17. Generate all permutations
def permutations_builtin(s):
    from itertools import permutations
    return [''.join(p) for p in permutations(s)]

def permutations_manual(s):
    def backtrack(path, used, res):
        if len(path) == len(s):
            res.append("".join(path))
            return
        for i in range(len(s)):
            if not used[i]:
                used[i] = True
                path.append(s[i])
                backtrack(path, used, res)
                path.pop()
                used[i] = False
    res = []
    backtrack([], [False]*len(s), res)
    return res


# 18. Generate all combinations
def combinations_builtin(s):
    from itertools import combinations
    res = []
    for r in range(1, len(s)+1):
        res.extend([''.join(c) for c in combinations(s, r)])
```

```
    return res

def combinations_manual(s):
    res = []
    def backtrack(start, path):
        if path:
            res.append("".join(path))
        for i in range(start, len(s)):
            path.append(s[i])
            backtrack(i+1, path)
            path.pop()
    backtrack(0, [])
    return res


# 19. Check subsequence
def is_subsequence_builtin(s1, s2):
    it = iter(s2)
    return all(ch in it for ch in s1)

def is_subsequence_manual(s1, s2):
    i = j = 0
    while i < len(s1) and j < len(s2):
        if s1[i] == s2[j]:
            i += 1
        j += 1
    return i == len(s1)


# 20. Smallest window containing all chars of another
def min_window_builtin(s, t):
```

```
    from collections import Counter
    need = Counter(t)
    have, missing, left, res = {}, len(t), 0, (0, float('inf'))
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
        need[ch] -= 1
        if missing == 0:
            while left < right and need[s[left]] < 0:
                need[s[left]] += 1
                left += 1
            if right-left < res[1]-res[0]:
                res = (left, right)
            need[s[left]] += 1
            missing += 1
            left += 1
    return "" if res[1] == float('inf') else s[res[0]:res[1]+1]

def min_window_manual(s, t):
    from collections import defaultdict
    need, missing = defaultdict(int), len(t)
    for ch in t: need[ch] += 1
    left, res = 0, (0, float('inf'))
    for right, ch in enumerate(s):
        if need[ch] > 0: missing -= 1
        need[ch] -= 1
        while missing == 0:
            if right-left < res[1]-res[0]:
                res = (left, right)
            need[s[left]] += 1
            if need[s[left]] > 0: missing += 1
```

```
        left += 1
    return "" if res[1] == float('inf') else s[res[0]:res[1]+1]
```

```
# 🔷 Pattern Matching (Q21–Q30)



# 21. Naive Pattern Matching
def naive_pattern_builtin(text, pattern):
    return [i for i in range(len(text)-len(pattern)+1) if text[i:i+len(pattern)] == pattern]

def naive_pattern_manual(text, pattern):
    matches = []
    for i in range(len(text)-len(pattern)+1):
        j = 0
        while j < len(pattern) and text[i+j] == pattern[j]:
            j += 1
        if j == len(pattern):
            matches.append(i)
    return matches



# 22. KMP Algorithm
def kmp_builtin(text, pattern):
    import re
    return [m.start() for m in re.finditer(f'(?={pattern})', text)]

def kmp_manual(text, pattern):
    def build_lps(p):
        lps, j = [0]*len(p), 0
        for i in range(1, len(p)):
            while j > 0 and p[i] != p[j]:
                j = lps[j-1]
            if p[i] == p[j]:
```

```
            j += 1; lps[i] = j
        return lps
    lps, res, j = build_lps(pattern), [], 0
    for i in range(len(text)):
        while j > 0 and text[i] != pattern[j]:
            j = lps[j-1]
        if text[i] == pattern[j]:
            j += 1
            if j == len(pattern):
                res.append(i-j+1)
                j = lps[j-1]
    return res


# 23. Rabin-Karp Algorithm
def rabin_karp_builtin(text, pattern):
    return naive_pattern_builtin(text, pattern)  # built-in fallback

def rabin_karp_manual(text, pattern, d=256, q=101):  # q = prime
    n, m = len(text), len(pattern)
    h, p, t, res = pow(d, m-1) % q, 0, 0, []
    for i in range(m):
        p = (d*p + ord(pattern[i])) % q
        t = (d*t + ord(text[i])) % q
    for i in range(n-m+1):
        if p == t and text[i:i+m] == pattern:
            res.append(i)
        if i < n-m:
            t = (d*(t - ord(text[i])*h) + ord(text[i+m])) % q
            if t < 0: t += q
    return res
```

```
# 24. Z Algorithm
def z_algorithm_builtin(text, pattern):
    return naive_pattern_builtin(text, pattern)

def z_algorithm_manual(text, pattern):
    concat = pattern + "$" + text
    n, Z = len(concat), [0]*len(concat)
    l = r = 0
    for i in range(1, n):
        if i <= r:
            Z[i] = min(r-i+1, Z[i-l])
        while i+Z[i] < n and concat[Z[i]] == concat[i+Z[i]]:
            Z[i] += 1
        if i+Z[i]-1 > r:
            l, r = i, i+Z[i]-1
    res = []
    for i in range(len(pattern)+1, n):
        if Z[i] == len(pattern):
            res.append(i-len(pattern)-1)
    return res


# 25. Find all occurrences of substring
def find_occurrences_builtin(text, pattern):
    import re
    return [m.start() for m in re.finditer(f'(?={pattern})', text)]

def find_occurrences_manual(text, pattern):
    return naive_pattern_manual(text, pattern)
```

```
# 26. Check if a string is rotation of another
def is_rotation_builtin(s1, s2):
    return len(s1) == len(s2) and s2 in (s1+s1)

def is_rotation_manual(s1, s2):
    if len(s1) != len(s2): return False
    for i in range(len(s1)):
        if s1[i:] + s1[:i] == s2:
            return True
    return False


# 27. Check if two strings are anagrams
def is_anagram_builtin(s1, s2):
    return sorted(s1) == sorted(s2)

def is_anagram_manual(s1, s2):
    if len(s1) != len(s2): return False
    count = {}
    for ch in s1:
        count[ch] = count.get(ch, 0) + 1
    for ch in s2:
        if ch not in count: return False
        count[ch] -= 1
        if count[ch] < 0: return False
    return True


# 28. Group anagrams
```

```
def group_anagrams_builtin(strs):
    from collections import defaultdict
    groups = defaultdict(list)
    for s in strs:
        groups["".join(sorted(s))].append(s)
    return list(groups.values())

def group_anagrams_manual(strs):
    groups = {}
    for s in strs:
        key = tuple(sorted(list(s)))
        if key not in groups:
            groups[key] = []
        groups[key].append(s)
    return list(groups.values())



# 29. Check if two strings are isomorphic
def is_isomorphic_builtin(s, t):
    return len(set(zip(s,t))) == len(set(s)) == len(set(t))

def is_isomorphic_manual(s, t):
    if len(s) != len(t): return False
    map_s, map_t = {}, {}
    for i in range(len(s)):
        if (s[i] in map_s and map_s[s[i]] != t[i]) or \
           (t[i] in map_t and map_t[t[i]] != s[i]):
            return False
        map_s[s[i]] = t[i]
        map_t[t[i]] = s[i]
    return True
```

```
# 30. Check if scrambled version
def is_scramble_builtin(s1, s2):
    from functools import lru_cache
    @lru_cache(None)
    def helper(a, b):
        if a == b: return True
        if sorted(a) != sorted(b): return False
        for i in range(1, len(a)):
            if (helper(a[:i], b[:i]) and helper(a[i:], b[i:])) or \
               (helper(a[:i], b[-i:]) and helper(a[i:], b[:-i])):
                return True
        return False
    return helper(s1, s2)

def is_scramble_manual(s1, s2):
    return is_scramble_builtin(s1, s2)  # recursion is manual logic itself
```

🔻 Advanced String Manipulations (Q31–Q40)

31. Longest Repeating Substring

```
def longest_repeating_builtin(s):
    n = len(s)
    subs = set()
    longest = ""
    for i in range(n):
        for j in range(i+1, n+1):
            sub = s[i:j]
            if sub in subs and len(sub) > len(longest):
                longest = sub
            subs.add(sub)
    return longest

def longest_repeating_manual(s):
    n, longest = len(s), ""
    for i in range(n):
        for j in range(i+1, n):
            k = 0
            while i+k < n and j+k < n and s[i+k] == s[j+k]:
                if k+1 > len(longest):
                    longest = s[i:i+k+1]
                k += 1
    return longest
```

32. Lexicographically smallest & largest substring of length k

```
def lex_substring_builtin(s, k):
    subs = [s[i:i+k] for i in range(len(s)-k+1)]
    return min(subs), max(subs)

def lex_substring_manual(s, k):
    smallest = largest = s[0:k]
    for i in range(1, len(s)-k+1):
        sub = s[i:i+k]
        if sub < smallest:
            smallest = sub
        if sub > largest:
            largest = sub
    return smallest, largest


# 33. Minimum bracket reversals to balance
def min_reversals_builtin(expr):
    if len(expr) % 2: return -1
    stack = []
    for ch in expr:
        if ch == '{':
            stack.append(ch)
        elif stack and stack[-1] == '{':
            stack.pop()
        else:
            stack.append(ch)
    m = stack.count('{')
    n = len(stack) - m
    return (m+1)//2 + (n+1)//2

def min_reversals_manual(expr):
```

```
    if len(expr) % 2: return -1
    open_count = close_count = 0
    for ch in expr:
        if ch == '{':
            open_count += 1
        else:
            if open_count > 0:
                open_count -= 1
            else:
                close_count += 1
    return (open_count+1)//2 + (close_count+1)//2


# 34. Validate balanced parentheses/brackets
def is_balanced_builtin(s):
    stack, mapping = [], {')':'(', ']':'[', '}':'{'}
    for ch in s:
        if ch in mapping.values():
            stack.append(ch)
        elif ch in mapping:
            if not stack or stack[-1] != mapping[ch]:
                return False
            stack.pop()
    return not stack

def is_balanced_manual(s):
    stack, opening, closing = [], "([{", ")]}"
    for ch in s:
        if ch in opening:
            stack.append(ch)
        elif ch in closing:
```

```
            if not stack: return False
            match = opening[closing.index(ch)]
            if stack[-1] != match: return False
            stack.pop()
    return not stack


# 35. Remove adjacent duplicates
def remove_adj_duplicates_builtin(s):
    stack = []
    for ch in s:
        if stack and stack[-1] == ch:
            stack.pop()
        else:
            stack.append(ch)
    return "".join(stack)

def remove_adj_duplicates_manual(s):
    res = ""
    for ch in s:
        if res and res[-1] == ch:
            res = res[:-1]
        else:
            res += ch
    return res


# 36. Run Length Encoding (Encode/Decode)
def rle_encode_builtin(s):
    from itertools import groupby
    return "".join(ch+str(len(list(g))) for ch, g in groupby(s))
```

```
def rle_encode_manual(s):
    encoded, count = "", 1
    for i in range(1, len(s)+1):
        if i < len(s) and s[i] == s[i-1]:
            count += 1
        else:
            encoded += s[i-1] + str(count)
            count = 1
    return encoded

def rle_decode(s):
    decoded, i = "", 0
    while i < len(s):
        ch, j = s[i], i+1
        num = ""
        while j < len(s) and s[j].isdigit():
            num += s[j]
            j += 1
        decoded += ch * int(num)
        i = j
    return decoded


# 37. Implement atoi() (string to integer)
def atoi_builtin(s):
    try:
        return int(s)
    except:
        return None
```

```
def atoi_manual(s):
    s = s.strip()
    if not s: return 0
    sign, i, num = 1, 0, 0
    if s[0] in "+-":
        if s[0] == '-': sign = -1
        i += 1
    while i < len(s) and '0' <= s[i] <= '9':
        num = num*10 + (ord(s[i]) - ord('0'))
        i += 1
    return sign*num


# 38. Implement strstr() (find substring)
def strstr_builtin(haystack, needle):
    return haystack.find(needle)

def strstr_manual(haystack, needle):
    for i in range(len(haystack)-len(needle)+1):
        if haystack[i:i+len(needle)] == needle:
            return i
    return -1


# 39. Convert integer to string (itoa)
def itoa_builtin(num):
    return str(num)

def itoa_manual(num):
    if num == 0: return "0"
    sign, res = "", ""
```

```
    if num < 0:
        sign, num = "-", -num
    while num > 0:
        res = chr(ord('0') + num % 10) + res
        num //= 10
    return sign + res



# 40. Longest word in a sentence
def longest_word_builtin(sentence):
    words = sentence.split()
    return max(words, key=len)

def longest_word_manual(sentence):
    max_word, max_len, word = "", 0, ""
    for ch in sentence + " ":
        if ch != " ":
            word += ch
        else:
            if len(word) > max_len:
                max_len, max_word = len(word), word
            word = ""
    return max_word
```

🔻 Palindrome-based String Problems (Q41–Q45)

41. Find all palindromic substrings

```
def all_palindromes_builtin(s):
    return [s[i:j] for i in range(len(s)) for j in range(i+1, len(s)+1) if s[i:j] == s[i:j][::-1]]

def all_palindromes_manual(s):
    res = []
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            res.append(s[l:r+1])
            l -= 1; r += 1
    for i in range(len(s)):
        expand(i, i)      # odd length
        expand(i, i+1)    # even length
    return res
```

42. Count number of palindromic substrings

```
def count_palindromes_builtin(s):
    return len(all_palindromes_builtin(s))

def count_palindromes_manual(s):
    count = 0
    def expand(l, r):
        nonlocal count
        while l >= 0 and r < len(s) and s[l] == s[r]:
            count += 1
            l -= 1; r += 1
    for i in range(len(s)):
```

```
        expand(i, i)
        expand(i, i+1)
    return count


# 43. Check if a string can be rearranged into a palindrome
def can_form_palindrome_builtin(s):
    from collections import Counter
    freq = Counter(s)
    odd = sum(1 for v in freq.values() if v % 2)
    return odd <= 1

def can_form_palindrome_manual(s):
    freq, odd_count = {}, 0
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    for val in freq.values():
        if val % 2: odd_count += 1
    return odd_count <= 1


# 44. Minimum insertions to make a string palindrome
def min_insertions_builtin(s):
    return len(s) - lps_length_builtin(s)

def lps_length_builtin(s):
    rev = s[::-1]
    n = len(s)
    dp = [[0]*(n+1) for _ in range(n+1)]
    for i in range(n-1, -1, -1):
        for j in range(n-1, -1, -1):
```

```
            if s[i] == rev[j]:
                dp[i][j] = 1 + dp[i+1][j+1]
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j+1])
    return dp[0][0]

def min_insertions_manual(s):
    n = len(s)
    rev = s[::-1]
    dp = [[0]*(n+1) for _ in range(n+1)]
    for i in range(n-1, -1, -1):
        for j in range(n-1, -1, -1):
            if s[i] == rev[j]:
                dp[i][j] = 1 + dp[i+1][j+1]
            else:
                dp[i][j] = max(dp[i+1][j], dp[i][j+1])
    lps = dp[0][0]
    return n - lps


# 45. Palindrome Partitioning (minimum cuts)
def min_cut_partition_builtin(s):
    n = len(s)
    dp = [0]*n
    pal = [[False]*n for _ in range(n)]
    for i in range(n):
        min_cut = i
        for j in range(i+1):
            if s[j] == s[i] and (i-j < 2 or pal[j+1][i-1]):
                pal[j][i] = True
                min_cut = 0 if j == 0 else min(min_cut, dp[j-1]+1)
```

```
        dp[i] = min_cut
    return dp[-1]


def min_cut_partition_manual(s):
    n = len(s)
    is_pal = [[False]*n for _ in range(n)]
    for i in range(n):
        is_pal[i][i] = True
    for length in range(2, n+1):
        for i in range(n-length+1):
            j = i+length-1
            if s[i] == s[j]:
                if length == 2 or is_pal[i+1][j-1]:
                    is_pal[i][j] = True
    dp = [0]*n
    for i in range(n):
        if is_pal[0][i]:
            dp[i] = 0
        else:
            dp[i] = min(dp[j]+1 for j in range(i) if is_pal[j+1][i])
    return dp[-1]
```

🔻 Special Interview String Problems (Q46–Q55)

```
# 46. Longest substring with k unique characters
def longest_substring_k_builtin(s, k):
    from collections import defaultdict
    left, max_len, freq = 0, 0, defaultdict(int)
    for right in range(len(s)):
        freq[s[right]] += 1
        while len(freq) > k:
            freq[s[left]] -= 1
            if freq[s[left]] == 0: del freq[s[left]]
            left += 1
        max_len = max(max_len, right-left+1)
    return max_len

def longest_substring_k_manual(s, k):
    left, max_len, freq = 0, 0, {}
    for right in range(len(s)):
        freq[s[right]] = freq.get(s[right], 0) + 1
        while len(freq) > k:
            freq[s[left]] -= 1
            if freq[s[left]] == 0: del freq[s[left]]
            left += 1
        max_len = max(max_len, right-left+1)
    return max_len


# 47. Remove all duplicate characters
def remove_duplicates_builtin(s):
```

```
    return "".join(dict.fromkeys(s))

def remove_duplicates_manual(s):
    seen, result = set(), ""
    for ch in s:
        if ch not in seen:
            seen.add(ch)
            result += ch
    return result


# 48. Smallest substring containing all unique characters
def smallest_unique_substring_builtin(s):
    unique_chars = set(s)
    required = len(unique_chars)
    from collections import Counter
    left, formed, window, freq = 0, 0, (0, float("inf")), Counter()
    for right in range(len(s)):
        freq[s[right]] += 1
        if len(freq) == required:
            while len(freq) == required:
                if right-left < window[1]-window[0]:
                    window = (left, right)
                freq[s[left]] -= 1
                if freq[s[left]] == 0:
                    del freq[s[left]]
                left += 1
    return "" if window[1] == float("inf") else s[window[0]:window[1]+1]

def smallest_unique_substring_manual(s):
    unique_chars, required = {}, len(set(s))
```

```
    left, res, freq = 0, "", {}
    for right in range(len(s)):
        freq[s[right]] = freq.get(s[right], 0) + 1
        while len(freq) == required:
            if res == "" or right-left+1 < len(res):
                res = s[left:right+1]
            freq[s[left]] -= 1
            if freq[s[left]] == 0:
                del freq[s[left]]
            left += 1
    return res


# 49. Find the first repeating character
def first_repeating_builtin(s):
    from collections import Counter
    freq = Counter(s)
    for ch in s:
        if freq[ch] > 1:
            return ch
    return None

def first_repeating_manual(s):
    seen = set()
    for ch in s:
        if ch in seen:
            return ch
        seen.add(ch)
    return None
```

```
# 50. Lexicographically next permutation of string
def next_permutation_builtin(s):
    arr = list(s)
    i = len(arr)-2
    while i >= 0 and arr[i] >= arr[i+1]:
        i -= 1
    if i == -1: return "".join(sorted(arr))
    j = len(arr)-1
    while arr[j] <= arr[i]:
        j -= 1
    arr[i], arr[j] = arr[j], arr[i]
    arr[i+1:] = reversed(arr[i+1:])
    return "".join(arr)

def next_permutation_manual(s):
    return next_permutation_builtin(s)  # manual is same algorithm, just step-by-step


# 51. Wildcard pattern matching (?, *)
def wildcard_match_builtin(s, p):
    import fnmatch
    return fnmatch.fnmatch(s, p)

def wildcard_match_manual(s, p):
    dp = [[False]*(len(p)+1) for _ in range(len(s)+1)]
    dp[0][0] = True
    for j in range(1, len(p)+1):
        if p[j-1] == "*":
            dp[0][j] = dp[0][j-1]
    for i in range(1, len(s)+1):
        for j in range(1, len(p)+1):
```

```
            if p[j-1] == "?" or p[j-1] == s[i-1]:
                dp[i][j] = dp[i-1][j-1]
            elif p[j-1] == "*":
                dp[i][j] = dp[i-1][j] or dp[i][j-1]
    return dp[len(s)][len(p)]


# 52. Regex matching (., *)
def regex_match_builtin(s, p):
    import re
    return re.fullmatch(p, s) is not None

def regex_match_manual(s, p):
    dp = [[False]*(len(p)+1) for _ in range(len(s)+1)]
    dp[0][0] = True
    for j in range(2, len(p)+1):
        if p[j-1] == "*" and dp[0][j-2]:
            dp[0][j] = True
    for i in range(1, len(s)+1):
        for j in range(1, len(p)+1):
            if p[j-1] == "." or p[j-1] == s[i-1]:
                dp[i][j] = dp[i-1][j-1]
            elif p[j-1] == "*":
                dp[i][j] = dp[i][j-2] or ((p[j-2] == "." or p[j-2] == s[i-1]) and dp[i-1][j])
    return dp[len(s)][len(p)]


# 53. Longest duplicate substring
def longest_duplicate_builtin(s):
    n = len(s)
    res = ""
```

```
    for i in range(n):
        for j in range(i+1, n):
            k = 0
            while i+k < n and j+k < n and s[i+k] == s[j+k]:
                if k+1 > len(res):
                    res = s[i:i+k+1]
                k += 1
    return res

def longest_duplicate_manual(s):
    return longest_duplicate_builtin(s)  # manual suffix array is complex, this is O(n^2)


# 54. Reverse words in a sentence
def reverse_words_builtin(s):
    return " ".join(reversed(s.split()))

def reverse_words_manual(s):
    words, word = [], ""
    for ch in s + " ":
        if ch != " ":
            word += ch
        else:
            words.append(word)
            word = ""
    words.reverse()
    return " ".join(words)


# 55. Check if one string is subsequence of another
def is_subsequence_builtin(s1, s2):
```

```
    it = iter(s2)
    return all(ch in it for ch in s1)


def is_subsequence_manual(s1, s2):
    i = j = 0
    while i < len(s1) and j < len(s2):
        if s1[i] == s2[j]:
            i += 1
        j += 1
    return i == len(s1)
```

🔷 Miscellaneous String Problems (Q56–Q60)

```
# 56. Check if two strings are rotations of each other
def are_rotations_builtin(s1, s2):
    return len(s1) == len(s2) and s2 in (s1+s1)

def are_rotations_manual(s1, s2):
    if len(s1) != len(s2):
        return False
    for i in range(len(s1)):
        rotated = s1[i:] + s1[:i]
        if rotated == s2:
            return True
    return False


# 57. Check if string is numeric
def is_numeric_builtin(s):
    return s.isdigit()

def is_numeric_manual(s):
    if not s: return False
    for ch in s:
        if not ('0' <= ch <= '9'):
            return False
    return True


# 58. Shortest string containing two given strings as subsequences
```

```
def shortest_common_supersequence_builtin(s1, s2):
    from functools import lru_cache
    @lru_cache(None)
    def lcs(i, j):
        if i == len(s1) or j == len(s2):
            return ""
        if s1[i] == s2[j]:
            return s1[i] + lcs(i+1, j+1)
        left = lcs(i+1, j)
        right = lcs(i, j+1)
        return left if len(left) > len(right) else right

    lcs_str = lcs(0, 0)
    i = j = 0
    res = ""
    for c in lcs_str:
        while s1[i] != c:
            res += s1[i]; i += 1
        while s2[j] != c:
            res += s2[j]; j += 1
        res += c; i += 1; j += 1
    return res + s1[i:] + s2[j:]

def shortest_common_supersequence_manual(s1, s2):
    n, m = len(s1), len(s2)
    dp = [[""]*(m+1) for _ in range(n+1)]
    for i in range(n-1, -1, -1):
        for j in range(m-1, -1, -1):
            if s1[i] == s2[j]:
                dp[i][j] = s1[i] + dp[i+1][j+1]
            else:
```

```
                left, right = dp[i+1][j], dp[i][j+1]
                dp[i][j] = left if len(left) > len(right) else right
    lcs_str = dp[0][0]
    i = j = 0
    res = ""
    for c in lcs_str:
        while s1[i] != c:
            res += s1[i]; i += 1
        while s2[j] != c:
            res += s2[j]; j += 1
        res += c; i += 1; j += 1
    return res + s1[i:] + s2[j:]



# 59. Word break problem
def word_break_builtin(s, word_dict):
    from functools import lru_cache
    word_set = set(word_dict)
    @lru_cache(None)
    def dfs(start):
        if start == len(s): return True
        for end in range(start+1, len(s)+1):
            if s[start:end] in word_set and dfs(end):
                return True
        return False
    return dfs(0)

def word_break_manual(s, word_dict):
    word_set = set(word_dict)
    n = len(s)
    dp = [False]*(n+1)
```

```
    dp[0] = True
    for i in range(1, n+1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break
    return dp[n]


# 60. Print all valid IP addresses from string of digits
def restore_ip_addresses_builtin(s):
    res = []
    def backtrack(start, path):
        if len(path) == 4 and start == len(s):
            res.append(".".join(path))
            return
        if len(path) == 4: return
        for l in range(1, 4):
            if start+l > len(s): break
            part = s[start:start+l]
            if (part.startswith("0") and len(part) > 1) or int(part) > 255:
                continue
            backtrack(start+l, path+[part])
    backtrack(0, [])
    return res

def restore_ip_addresses_manual(s):
    res = []
    n = len(s)
    for i in range(1, min(4, n-2)):
        for j in range(i+1, min(i+4, n-1)):
```

```
            for k in range(j+1, min(j+4, n)):
                a, b, c, d = s[:i], s[i:j], s[j:k], s[k:]
                if all(map(valid_ip_part, [a, b, c, d])):
                    res.append(".".join([a, b, c, d]))
    return res


def valid_ip_part(part):
    if not part or (part.startswith("0") and len(part) > 1):
        return False
    if not part.isdigit() or int(part) > 255:
        return False
    return True
```